

Programación Dinámica I

Teoría de Algoritmos (TB024, 75.29, 95.06)
Curso Echevarría

Programación Dinámica: Conceptos

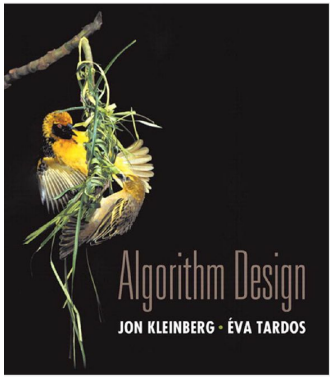


Cottonbro Studio

Programación Dinámica: Características

- Es una técnica de diseño de algoritmos, tal como puede ser D&C, Greedy o Backtracking.
- Desarrollada por Richard Bellman (el mismo del algoritmo de Bellman)
- Según nos cuenta Dreyfus (2002):
  - Programación: refiere a planificación, toma de decisiones
  - Dinámica: refiere a que varía en el tiempo, es multietapa. Y es un adjetivo que no puede usarse peyorativamente.

Dreyfus, S. (2002). "Richard Bellman on the Birth of Dynamic Programming". Institute of Operations Research and Management Science (INFORMS)



Programación Dinámica: Características (cont.)

- Se parece a D&C en que divide el problema en partes más pequeñas
- Se diferencia de D&C en que tiene un enfoque Bottom-Up
  - Bottom-Up: toma el caso base y lo usa para resolver subproblemas cada vez más grandes hasta llegar al problema a resolver
  - Top-Down (D&C): toma el problema a resolver y lo divide en subproblemas cada vez más chicos hasta llegar al caso base
- Es especialmente útil para resolver problemas de optimización
- Los problemas que se resuelven por Programación Dinámica también pueden resolverse recursivamente
- Se puede usar una Relación de Recurrencia para establecer su complejidad temporal

Programación Dinámica: 2 requisitos

1. Subestructura Óptima:
   - La solución óptima del problema puede obtenerse a partir de la solución óptima de sus subproblemas
   - La solución óptima contiene a las soluciones óptimas de los subproblemas
   - Generalmente se describen recursivamente
   - Los algoritmos Greedy también comparten este requisito
   - Cuidado: la solución a un subproblema *no debe afectar* a la solución de otro subproblema
2. Subproblemas Superpuestos:
   - En la solución de los subproblemas vuelve a aparecer la solución de otros subproblemas resueltos previamente, y se reutilizan
   - En D&C los subproblemas no se solapan; en Programación Dinámica, sí

Programación Dinámica: Memoization

Memorización, Memoización o Memoization:

- Técnica para mejorar la eficiencia de Programación Dinámica
  - Lo logra recordando resultados para reducir la cantidad de subproblemas a resolver
- Se trata de almacenar los resultados de subproblemas que volveré a usar más tarde
- Ofrece un condimento “Top-Down” para mejorar la base “Bottom-Up”
- Requiere del principio de Subproblemas Superpuestos

Fibonacci Dinámico



Janiere Fernandez

Fibonacci: Versión Recursiva

```
def fib(n)
  if n<=1 return n
  return fib(n-1)+fib(n-2)
```

¿Cuántas veces llamamos recursivamente para valores que ya calculamos anteriormente?



Fibonacci: Versión Recursiva con Memoization

```
def fib_con_memoria(n):
  M_FIB = [None] * (n+1)
  return fib_memoized(n, M_FIB)

def fib_memoized(n, M_FIB):
  if n == 0:
    return 0
  if n == 1:
    return 1
  if M_FIB[n-1] == None:
    M_FIB[n-1] = fib_memoized(n-1, M_FIB)
  if M_FIB[n-2] == None:
    M_FIB[n-2] = fib_memoized(n-2, M_FIB)
  M_FIB[n] = M_FIB[n-1] + M_FIB[n-2]
  return M_FIB[n]
```

Fibonacci: Versión Iterativa + Memoization

```
def fib_iterativo(n):
  M_FIB = [None] * (n+1)
  M_FIB[0] = 0
  M_FIB[1] = 1
  for i in range(2, n+1):
    M_FIB[i] = M_FIB[i-1] + M_FIB[i-2]
  return M_FIB[n]
```

¿Realmente hace falta mantener un Array de tamaño n sólo para usar los últimos dos valores?

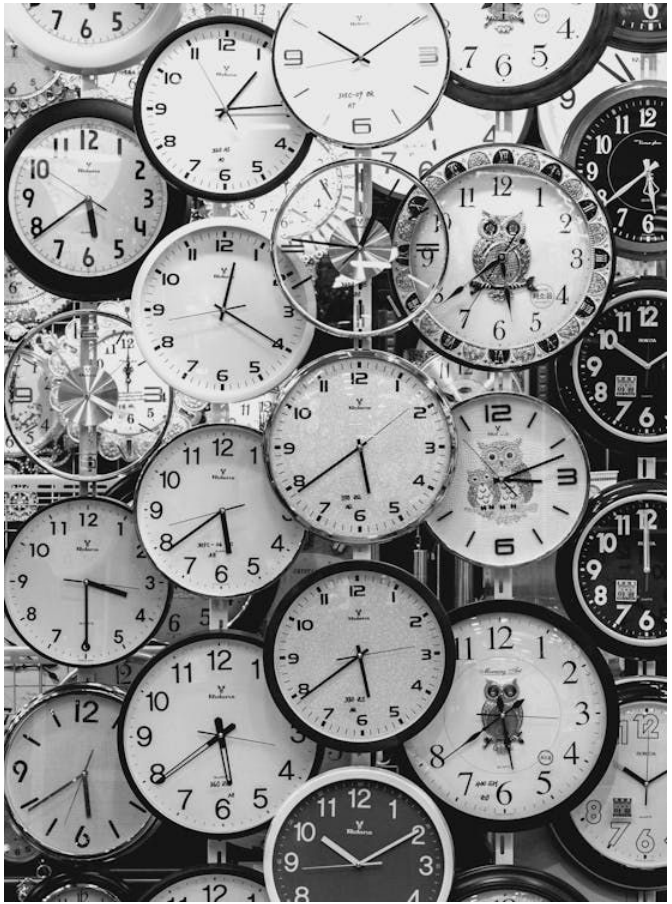
Fibonacci: Versión Programación Dinámica

```
def fib_dinamico(n):
  if n == 0:
    return 0
  if n == 1:
    return 1
  anterior = 0
  actual = 1
  for i in range(1, n):
    nuevo = actual + anterior
    anterior = actual
    actual = nuevo
  return actual
```



Planificación de Intervalos Ponderados

(Weighted Interval Scheduling)



Andrey Grushnikov

Weighted Interval Scheduling: Definición

Recordemos Interval Scheduling (Clase I de Greedy):

- Tenemos un conjunto de tareas $S=\{1,2,\ldots, n\}$
- La $i\text{-}ésima$ tarea empieza en el horario s_i y termina en el horario f_i (y no se pueden modificar ni interrumpir). La duración d_i es $d_i=f_i-s_i$
- Definición: un subconjunto de tareas es compatible si no hay solapamiento entre ellas

Encontrar el subconjunto compatible más grande posible

Weighted Interval Scheduling: Definición (cont.)

Ahora definamos Weighted Interval Scheduling:

- Tenemos un conjunto de tareas $S=\{1,2,\ldots, n\}$
- La $i\text{-}ésima$ tarea empieza en el horario s_i y termina en el horario f_i (y no se pueden modificar ni interrumpir). La duración d_i es $d_i=f_i-s_i$
- Definición: un subconjunto de tareas es compatible si no hay solapamiento entre ellas
- Cada tarea tiene un peso w_i

Encontrar el subconjunto T compatible que maximice el peso total W

Weighted Interval Scheduling: Ejemplo

```
1)  [----]                  w1=1
2)     [-------]            w2=3
3)        [----]            w3=1
```

Si resolvemos con el mismo algoritmo greedy que vimos anteriormente:

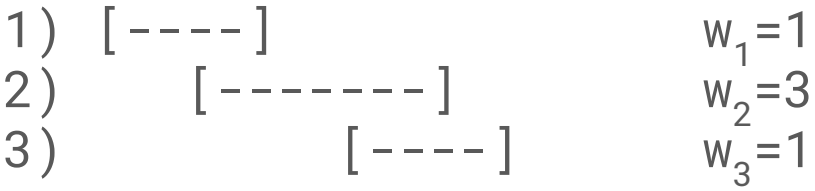
- $T=\{1,3\}; W=2$

Pero la solución óptima es:

- $T=\{2\}; W=3$



Weighted Interval Scheduling: Ejemplo (cont.)

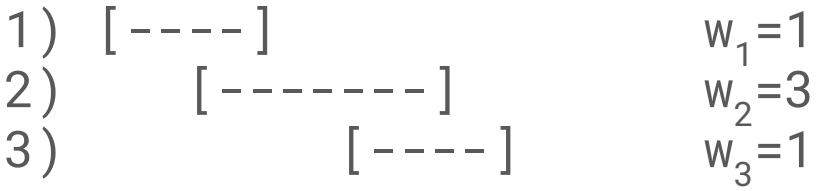


¿Cómo se resolvería por Fuerza Bruta?

Explora todas las posibilidades:

- $T=\{1, 2, 3\}$; incompatible
- $T=\{1,2\}$; incompatible
- $T=\{1,3\}$; $W=2$
- $T=\{1\}$; $W=1$
- $T=\{2,3\}$; incompatible
- $T=\{2\}$; $W=3$
- $T=\{3\}$; $W=1$
- $T=\{\}$; $W=0$

Weighted Interval Scheduling: Ejemplo (cont.)



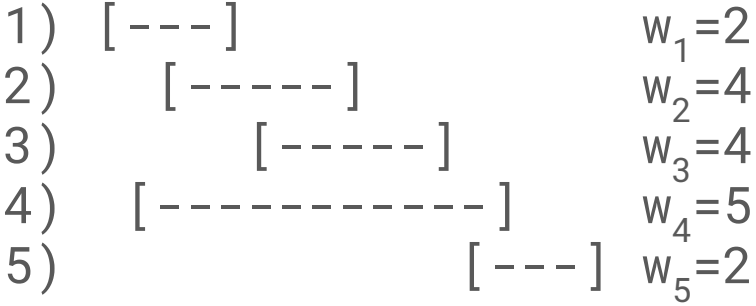
¿Cómo se resolvería por Backtracking?

- Construimos progresivamente candidatos parciales
- Podamos cuando encontramos elementos incompatibles entre sí

Weighted Interval Scheduling: Análisis

- Ordenemos los intervalos por menor tiempo de finalización f_i
  - Si $i<j$ entonces $f_i \leq f_j$
- Definamos una función $P(k)$ como el mayor índice $i<k$ tal que los elementos i y k son disjuntos, o 0 si no existe ningún elemento $i<k$ disjunto con k
- Definamos T_k como una solución óptima que puede incluir (o no) los primeros k elementos de la lista ordenada de intervalos
- Definamos una función $W(k)$ que devuelve el peso de la solución óptima T_k

Weighted Interval Scheduling: Análisis (cont.)



¿Cuánto valen T_k, $W(k)$ y $P(k)$ para $k=\{0,1,..,5\}$?

- $T_0=\{\}$; $W(0)=0$; $P(0)=0$
- $T_1=\{1\}$; $W(1)=2$; $P(1)=0$
- $T_2=\{2\}$; $W(2)=4$; $P(2)=0$
- $T_3=\{1,3\}$; $W(3)=6$; $P(3)=1$
- $T_4=\{1,3\}$; $W(4)=6$; $P(4)=0$
- $T_5=\{1,3,5\}$; $W(5)=8$; $P(5)=3$

Weighted Interval Scheduling: Análisis (cont.)

Para cada k, T_k puede incluir o no los primeros k elementos de la lista ordenada de intervalos:

- Podría incluir el *k-ésimo* elemento:
  - Entonces ningún elemento entre $P(k)+1$ y k puede pertenecer a T_k porque se solaparían
  - Tampoco se solapan k y los elementos $\{1,..,\ P(k)\}$ incluidos en T_k
- Podría no incluir el *k-ésimo* elemento:
  - Entonces T_k es una solución formada por elementos de $\{1,..,\ k\text{-}1\}$ que no se solapan

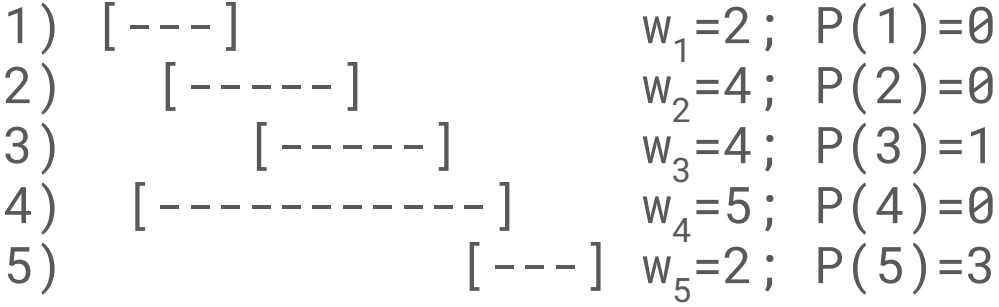
Weighted Interval Scheduling: Análisis (cont.)

```
1) [---]                  w1=2; P(1)=0
2)   [-----]              w2=4; P(2)=0
3)      [-----]           w3=4; P(3)=1
4)  [-----------]         w4=5; P(4)=0
5)              [---]     w5=2; P(5)=3
```

Analicemos $k=5$

- $T_k=\{1, 3, 5\}$
- El elemento $k=5$ está en T_k
- $P(5)=3$
- Los elementos entre $P(5)+1$ y 5 no están en T_k
- Los elementos entre 1 y $P(5)$ incluidos en T_k no se solapan

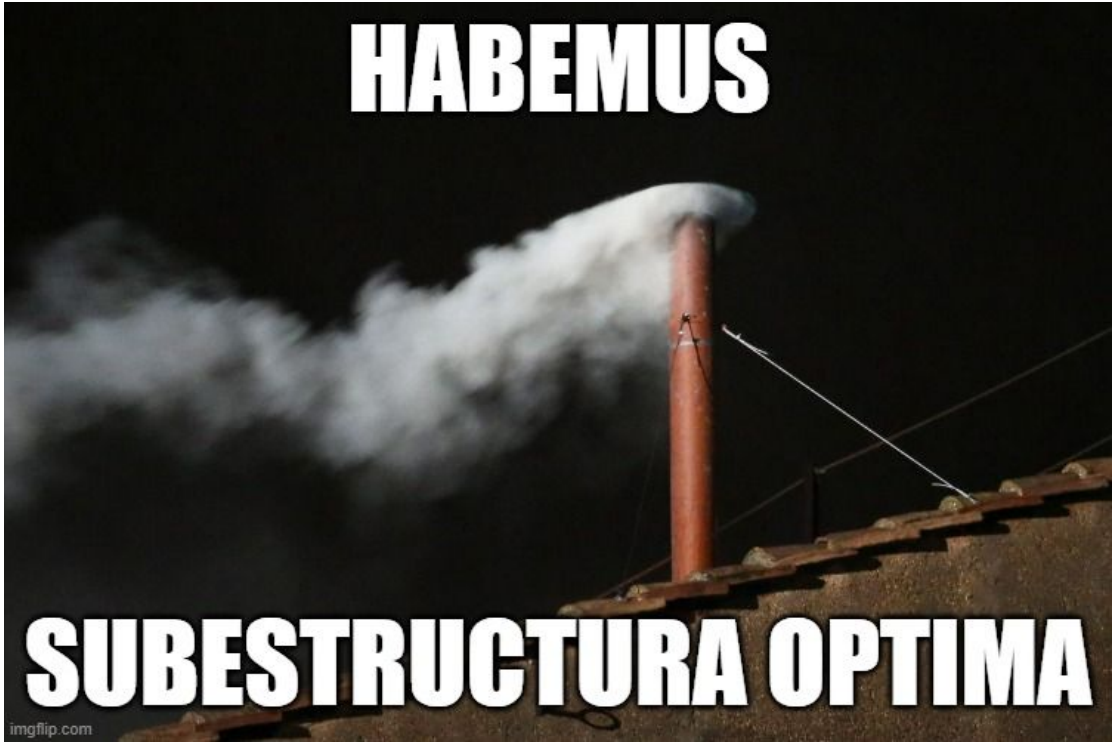
Weighted Interval Scheduling: Análisis (cont.)



Analicemos $k=4$

- $T_k=\{1, 3\}$
- El elemento $k=4$ no está en T_k
- $P(4)=0$
- Los elementos entre 1 y $k-1$ incluidos en T_k no se solapan

Weighted Interval Scheduling: Análisis (cont.)



Weighted Interval Scheduling: Análisis (cont.)

Para todo elemento k, el valor de $W(k)$ dependerá de si el k-*ésimo* elemento está o no en T_k:

- Si el k-*ésimo* elemento no está en T_k:
  - $W(k) = W(k\text{-}1)$
- Si el k-*ésimo* elemento está en T_k:
  - $W(k) = w_k + W(P(k))$

Weighted Interval Scheduling: Análisis (cont.)

```
1) [---]                  w1=2; P(1)=0
2)   [-----]              w2=4; P(2)=0
3)      [-----]           w3=4; P(3)=1
4)  [-----------]         w4=5; P(4)=0
5)             [---]      w5=2; P(5)=3
```

Analicemos $k=4$

- $T_k=\{1, 3\}$
- El elemento $k=4$ no está en T_k
- $W(4)=W(3)=6$

Weighted Interval Scheduling: Análisis (cont.)

```
1) [---]                  w1=2; P(1)=0
2)   [-----]              w2=4; P(2)=0
3)      [-----]           w3=4; P(3)=1
4)  [-----------]         w4=5; P(4)=0
5)              [---]     w5=2; P(5)=3
```

Analicemos $k=5$

- $T_k=\{1, 3, 5\}$
- El elemento $k=5$ está en T_k
- $W(5)=w_5 + W(P(5))=w_5+W(3)=2+6=8$

Weighted Interval Scheduling: Análisis (cont.)

Entonces podemos asegurar que:

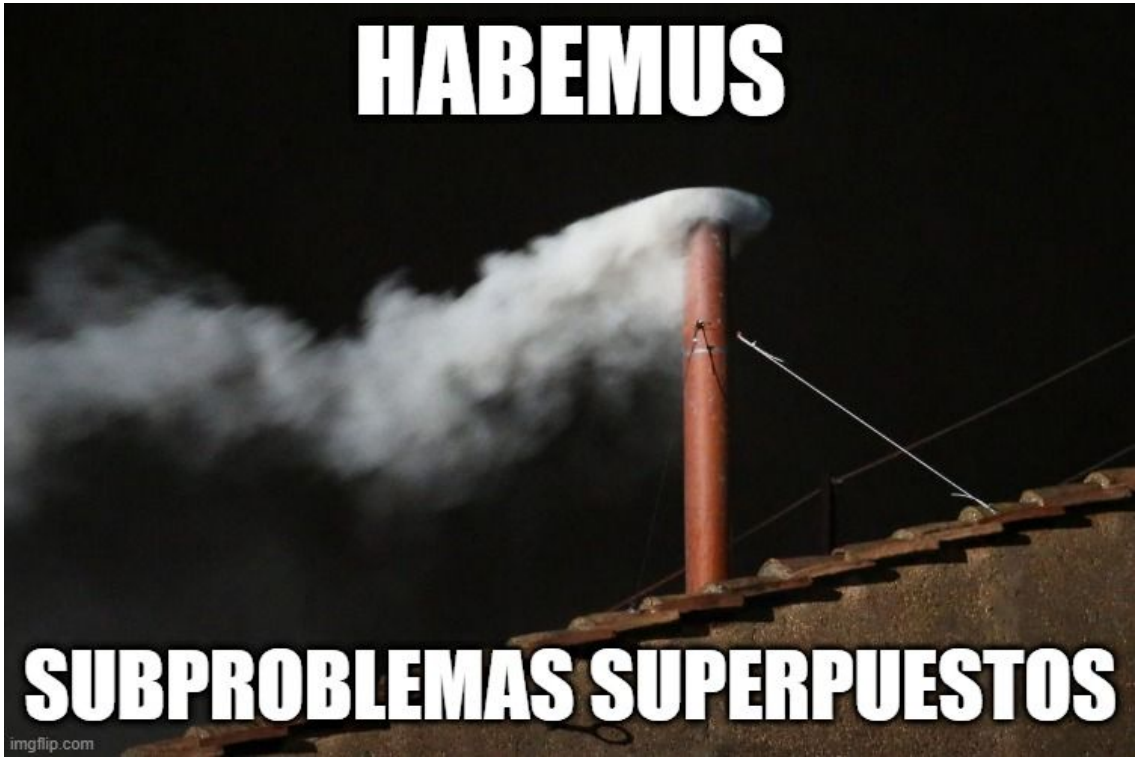
$$W(k)=max(w_k + W(P(k)), W(k-1))$$

Y además, todo elemento k estará en T_k si y sólo si:

$$w_k + W(P(k)) \geq W(k-1))$$



Weighted Interval Scheduling: Análisis (cont.)



Weighted Interval Scheduling: Recursivo

```
def w(k, p, pesos):
  if k==0: return 0
  return max(pesos[k]+w(p[k],p,pesos), w(k-1,p,pesos))
```

Weighted Interval Scheduling: +Memoization

```
def w_con_memoria(k, p, pesos):
  M = [None] * (k+1)
  return w_memoized(k, p, pesos, M)

def w_memoized(k, p, pesos, M):
  if k == 0:
    return 0
  if M[k-1] == None:
    M[k-1] = w_memoized(k-1, p, pesos, M)
  M[k] = max(pesos[k] + M[p[k]], M[k-1])
  return M[k]
```

Weighted Interval Scheduling: Dinámico

```
def w_dinamico(p, pesos):
  n = len(pesos)
  M = [None] (n +1)
  M[0] = 0

  for k in range(1, n+1):
    M[k]=max(pesos[k] + M[p[k]], M[k-1])
  return M[n]
```

Weighted Interval Scheduling: Obtener Solución

```
def t_dinamico(k, p, pesos, M, solucion):
  if k==0: return solucion
  if pesos[k] + M[p[k]] >= M[k-1]:
    solucion.append(k)
    return t_dinamico(p[k], p, pesos, M, solucion)
  else:
    return t_dinamico(k-1, p, pesos, M, solucion)
```

Weighted Interval Scheduling: Complejidad

Para la versión Backtracking/Fuerza Bruta:

- $O(2^n)$

Para la versión Dinámica:

- $O(n \log n)$
  - Ordenar por fecha fin: $O(n \log n)$
  - Obtener $P(k)$: $O(n \log n)$ (*)
  - Core Dinámico: $O(n)$

(*) Encontrar $P(k)$ para un elemento es $O(\log n)$ por búsqueda binaria. Encontrar $P(k)$ para todos los elementos es $O(n \log n)$.

Corte de Varilla



binary Ego

Corte de Varilla: Definición

- Se tiene una varilla de longitud n
- Se puede cortar (o no) en cualquier cantidad de cortes de longitud $1 \leq i \leq n$
- Cortar no tiene costo
- El precio de venta de un corte de tamaño i es p_i

Se quiere encontrar la manera de cortar la varilla y vender los cortes que maximice la ganancia total g_n

Corte de Varilla: Definición (cont.)

- Tanto la longitud como los cortes son enteros positivos
- Una posible solución sería no hacer cortes y vender la varilla entera, de longitud n
- Se podrían hacer cortes de cualquier longitud $1 \leq i \leq n$
- Se podrían hacer algunos cortes más de una vez
- Hasta se podrían hacer algunos cortes sí y otros no



Corte de Varilla: Definición (cont.)

Ejemplo:

- Si la varilla mide $n=4$, hay $2^{n-1}=8$ soluciones posibles:

```
1)       [====]        g4=9
2)     [=] [===]       g4=9
3)     [==] [==]       g4=10
4)     [===] [=]       g4=9
5)   [=] [=] [==]      g4=7
6)   [=] [==] [=]      g4=7
7)   [==] [=] [=]      g4=7
8) [=] [=] [=] [=]     g4=4
```

Precios de Venta:

```
[====]  p4=9
[===]   p3=8
[==]    p2=5
[=]     p1=1
```

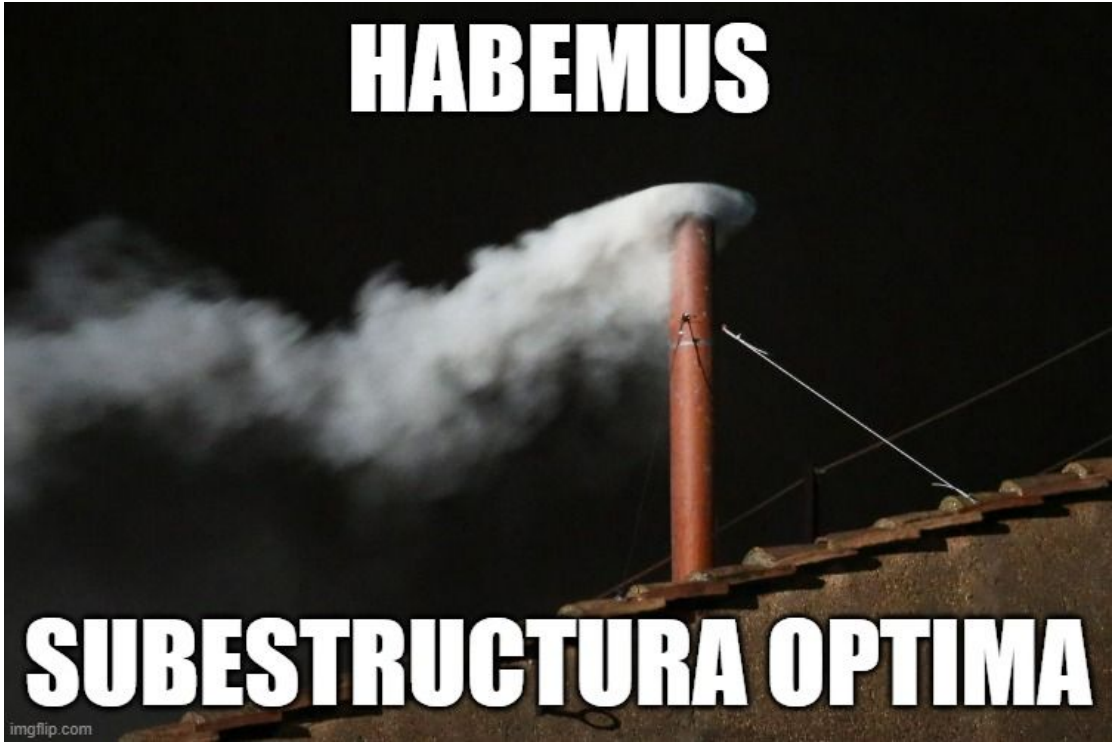
Corte de Varilla: Análisis

- Definimos g_i como la mayor ganancia que se puede obtener de una varilla de longitud i $(1 \leq i \leq n)$
- Como no conocemos la forma óptima de cortar, probamos todas las posibilidades:
  - Empezamos con un corte $i=1$, que tiene un precio p_1
  - Y buscamos la forma óptima de cortar el resto de la varilla, que mide $n-1$
  - Después, probamos un corte $i=2$, con un precio p_2
  - Y buscamos la forma óptima de cortar el resto de la varilla, que mide $n-2$
  - Etc.
- Al final, nos quedamos con la mejor solución:
  - $g_n = max\{p_n, p_1 + g_{n-1}, p_2 + g_{n-2}, \ldots, p_{n-1} + g_1\}$

Corte de Varilla: Análisis (cont.)

- Para resolver el problema de tamaño n, debemos resolver problemas idénticos pero más chicos
- Hecho el primer corte, las dos partes forman dos subproblemas idénticos al original (pero más chicos)

Corte de Varilla: Análisis (cont.)



Corte de Varilla: Recursivo

```
def g(p, n):
  if n==0: return 0
  q=-1
  for i in range(1,n+1):
    q=max(q, p[i] + g(p, n-i))
  return q
```

Corte de Varilla: Recursivo (cont.)

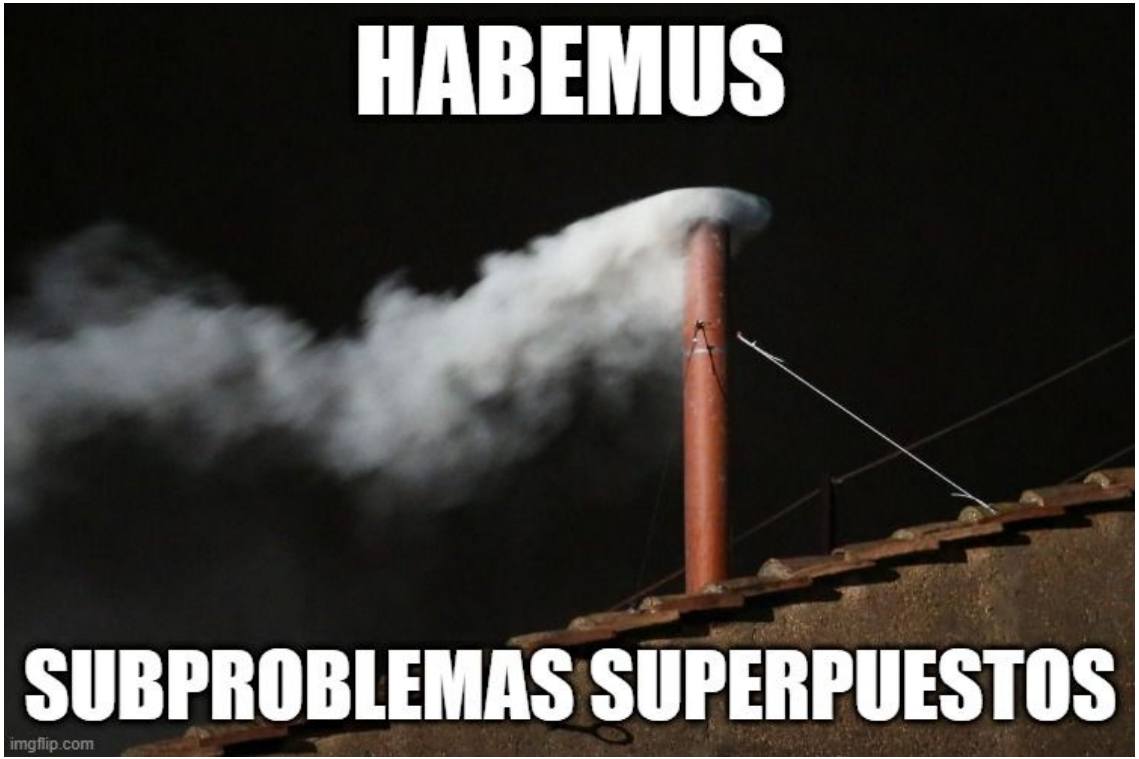
Costo de la variante recursiva:

- El algoritmo se invoca recursivamente una y otra vez con los mismos valores de n
- La cantidad de llamadas recursivas para un n cualquiera es igual a la cantidad de nodos del subárbol cuya raíz es n

$$\left.\begin{array}{r} T(0) = 1 \\ T(n) = 1 + \sum_{j=0}^{n-1} T(j) \end{array}\right\} \Leftrightarrow T(n) = 2^n$$



Corte de Varilla: Recursivo (cont.)



Corte de Varilla: Recursivo + Memoization

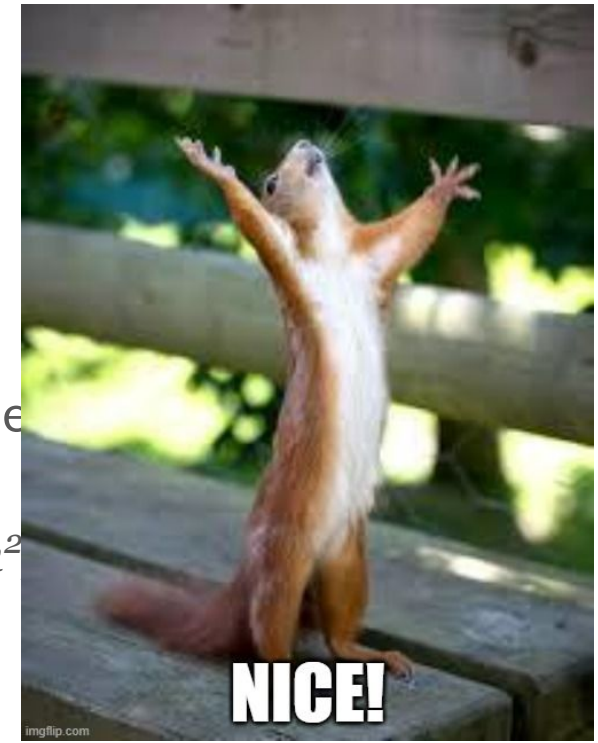
```
def g_con_memoria(p, n):
  M = [-1] * (n+1)
  return g_memoized(p, n, M)

def g_memoized(p, n, M):
  if M[n] != -1:
    return M[n]
  if n == 0:
    return 0
  q=-1
  for i in range(1,n+1)
    q=max(q, p[i] + g_memoized(p, n-i, M))
  M[n]=q
  return q
```

Corte de Varilla: Recursivo + Memoization (cont.)

Costo de la variante Recursiva con Memoization:

- La primera iteración itera n veces
- La segunda, n-1 veces
- La tercera, n-2 veces
- …
- El total de las llamadas recursivas es la suma de serie aritmética: $\frac{n(n-1)}{2}$
- La variante recursiva con Memoization es $O(n^2$



Corte de Varilla: Dinámico

```
def g_dinamico(p, n):
  M = [-1] * (n+1)
  M[0]=0

  for j in range(1,n+1): # tamaño de la varilla
    q=-1
    for i in range(1,j+1): # posición del corte
      q=max(q, p[i] + M[j-i])
    M[j]=q
  return M[n]
```

Corte de Varilla: Dinámico (cont.)

Costo de la variante Dinámica

- Hay dos loops anidados:
  - El loop externo itera n veces
  - El loop interno es una serie aritmética
- La variante Dinámica también es $O(n^2)$



Corte de Varilla: Obtener la Solución

```
def g_dinamico_y_solucion(p, n, solucion):
  M = [-1] * (n+1)
  M[0]=0

  for j in range(1,n+1): # tamaño de la varilla
    q=-1
    for i in range(1,j+1): # posición del corte
      if q<p[i] + M[j-i]
        q=p[i] + M[j-i]
   solucion[j]=i # guardar la posición del corte
    M[j]=q
  return M[n]
```



¡Muchas gracias!